Brain Tumor Detector — Complete Project Documentation

Expert-level MRI brain tumor classification using EfficientNetB4 + Flask 4 classes:
Glioma · Meningioma · No Tumor · Pituitary

Table of Contents

1. Project Overview
2. Project Structure
3. Architecture Decisions
4. Data Pipeline
5. Augmentation Pipeline
6. Model Architecture
7. Training Strategy
8. Grad-CAM Explainability
9. Flask Web Application
10. Evaluation & Metrics
11. Bugs Fixed & Changes Made
12. How to Run

---

1. Project Overview

This project classifies brain MRI scans into four categories using deep learning. It is designed for educational and research purposes, **not** clinical diagnosis.

Item	Detail
Task	4-class image classification
Classes	`glioma`, `meningioma`, `no_tumor`, `pituitary`
Backbone	EfficientNetB4 (ImageNet pretrained)
Input Size	380 × 380 × 3
Framework	TensorFlow / Keras
Web App	Flask
Explainability	Grad-CAM heatmaps

---

2. Project Structure

```
brain-tumor-detector/
│
├── app.py                          # Flask web application (inference server)
│
```

```
├── notebooks/
│   ├── train.ipynb              # Original (basic) training notebook
│   └── train_v2.ipynb           # Expert training notebook (current)
│
├── utils/
│   ├── augmentation.py          # MRI-specific augmentation & normalization
│   ├── preprocess.py            # Flask inference preprocessing pipeline
│   ├── gradcam.py               # Grad-CAM heatmap generation
│   └── evaluation.py            # Metrics, plots, classification report
│
├── models/
│   ├── best_model.keras         # Saved best checkpoint (after training)
│   ├── label_map.json           # Class name → index mapping
│   └── model_info.json          # Training metadata
│
├── data/
│   ├── train/                   # Training images (4 class subdirectories)
│   ├── val/                     # Validation images
│   └── test/                    # Test images (evaluate only once)
│
├── templates/
│   ├── index.html               # Landing page
│   ├── predict.html             # Upload form
│   └── result.html              # Prediction result + Grad-CAM
│
├── static/                      # CSS, JS, static assets
├── reports/                     # Auto-generated plots and reports
├── logs/                        # CSV training logs
├── scripts/
│   └── download_data.py         # Dataset download helper
├── requirements.txt
└── README.md
```

---

3. Architecture Decisions

Why EfficientNetB4?

EfficientNetB4 was chosen over simpler alternatives for several reasons:

- **Native 380×380 input** — matches the resolution MRI images need to capture fine-grained tumor boundaries
- **Compound scaling** — simultaneously scales depth, width, and resolution, making it more parameter-efficient than VGG or ResNet
- **ImageNet pretraining** — low-level feature detectors (edges, textures, shapes) transfer well to MRI even though MRI looks different from natural images
- **Better than B0/B1** for a 4-class medical imaging task without GPU memory issues

Why NOT use `preprocess_input`?

EfficientNet's built-in `preprocess_input` applies ImageNet mean/std normalization designed for natural images. MRI scans have fundamentally different intensity distributions (scanner-dependent, not globally calibrated). Instead, this project uses:

1. **CLAHE** — adaptive contrast enhancement tuned for MRI
2. **Per-image Z-score normalization** — handles scanner intensity variation

Why Mixed Precision?

`mixed_float16` stores weights in float16 but accumulates gradients in float32. This gives: - 2–3× faster training on Tensor Core GPUs (RTX/A-series) - ~50% less GPU memory usage - The output `Dense` layer is forced to `dtype='float32'` to preserve numerical stability in softmax

---

4. Data Pipeline

Loading (`notebooks/train_v2.ipynb` — Cell 3)

```
tf.keras.utils.image_dataset_from_directory(
    directory,
    image_size=(380, 380),
    batch_size=16,
    label_mode='categorical',
    color_mode='rgb',     # forces 3-channel even for grayscale MRI files
    class_names=['glioma', 'meningioma', 'no_tumor', 'pituitary'],
)
```

`color_mode='rgb'` is critical — some MRI PNGs in the dataset are saved as single-channel grayscale. Without this, they enter the pipeline as 1-channel tensors, crashing augmentation functions that expect 3 channels.

Pipeline Order

Split	Pipeline
Train	raw → `.cache()` → `.map(augment)` → `.shuffle(1024)` → `.prefetch()`
Val / Test	raw → `.map(preprocess_eval)` → `.cache()` → `.prefetch()`

Why cache before augment for train? Caching the raw images avoids re-reading from disk every epoch. Augmentation is applied after the cache, so each epoch sees different random augmentations of the same images.

Class Imbalance

Class weights are computed using `sklearn.utils.class_weight.compute_class_weight('balanced', ...)` and passed to

`model.fit(class_weight=...)`. This penalizes mistakes on minority classes more heavily during training.

---

5. Augmentation Pipeline

File: `utils/augmentation.py`

Always Applied (Train + Inference)

1. CLAHE — `apply_clahe()`

Contrast Limited Adaptive Histogram Equalization applied per-channel.

- Converts float32 [0,1] → uint8 [0,255]
- Splits R, G, B channels
- Applies `cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))` to each channel independently
- Merges and converts back to float32 [0,1]

Why CLAHE for MRI? MRI images often have low local contrast — small tumor regions may have very similar intensity to surrounding tissue. CLAHE enhances local contrast without over-amplifying noise (unlike global histogram equalization).

2. Z-Score Normalization — `zscore_normalize()`

```
normalized = (image - mean) / (std + 1e-7)
clipped    = clip(normalized, -3, 3)
output     = (clipped + 3) / 6         → maps to [0, 1]
```

Why Z-score instead of /255? MRI scanner intensity is not globally calibrated. Dividing by 255 treats all scanners as if they produce images with the same intensity range — they don't. Z-score normalization is per-image and handles this variation.

Applied During Training Only (Probabilistic)

Augmentation	Probability	Purpose
Gaussian Noise (σ=0.015)	50%	Simulates MRI thermal noise
Random Gaussian Blur (σ≤0.8)	30%	Simulates scanner resolution variation
Elastic Deformation (α=25, σ=4)	40%	Simulates natural brain shape variation
Brightness/Contrast Jitter (±0.12)	40%	Simulates inter-scanner intensity differences

Elastic Deformation — `elastic_transform()`

Generates a smooth random displacement field by: 1. Sampling a random [H, W] displacement map in [-1, 1] 2. Smoothing with a Gaussian kernel (σ=4) to make

deformations locally coherent 3. Scaling by `alpha=25` to control magnitude 4. Remapping the image using `cv2.remap` with reflect border padding

This is the most medically realistic augmentation — real brain anatomy varies subtly in shape between patients.

Grayscale Safety Guards

Both `mri_augment()` and `apply_clahe()` contain explicit grayscale → RGB conversion:

```
# In mri_augment:
if image_np.ndim == 2:
    image_np = np.stack([image_np] * 3, axis=-1)
elif image_np.shape[-1] == 1:
    image_np = np.concatenate([image_np] * 3, axis=-1)

# In apply_clahe:
if img_uint8.ndim == 2:
    img_uint8 = np.stack([img_uint8] * 3, axis=-1)
elif img_uint8.ndim == 3 and img_uint8.shape[2] == 1:
    img_uint8 = np.concatenate([img_uint8] * 3, axis=-1)
```

This protects the Flask inference path where images might arrive as grayscale through any route.

---

6. Model Architecture

File: `notebooks/train_v2.ipynb` — Cell 8

```
Input (380, 380, 3)
    │
    ▼
EfficientNetB4 backbone (ImageNet pretrained)
    │  – Frozen in Phase 1
    │  – Progressively unfrozen in Phases 2–5
    │  – BatchNormalization layers always kept frozen
    ▼
GlobalAveragePooling2D
    ▼
Dense(512) → BatchNorm → ReLU → Dropout(0.4)
    ▼
Dense(256) → BatchNorm → ReLU → Dropout(0.3)
    ▼
Dense(4, activation='softmax', dtype='float32')
```

Key Design Choices

- **No Rescaling layer inside the model** — preprocessing happens in the data pipeline, not the model. This keeps the model's `predict()` consistent with training.

- **BatchNormalization before Activation** — standard pattern for modern classification heads, more stable than Dense → ReLU → BN.
- **L2 regularization (λ=1e-4)** on all Dense layers — prevents overfitting on the relatively small medical dataset.
- **Label smoothing = 0.1** — the model is trained with soft targets (0.025 / 0.925 instead of 0 / 1), which prevents overconfident predictions and improves calibration.

---

7. Training Strategy

5-Phase Gradual Unfreezing

Gradual unfreezing is the most important technique for fine-tuning pretrained models on small datasets. Unfreezing all layers at once destroys the ImageNet features.

Phase	Epochs	LR	Backbone
1	15	1e-3	Fully frozen
2	15	1e-4	Top 25% unfrozen
3	15	5e-5	Top 50% unfrozen
4	15	2e-5	Top 75% unfrozen
5	10	5e-6	Fully unfrozen (BN frozen)

BatchNormalization layers stay frozen throughout all phases. Unfreezing BN layers causes them to update their running statistics to match the small training batch, destroying the ImageNet statistics that make transfer learning work.

Learning Rate Schedule: Warmup + Cosine Decay

```
LR
│     /\
│    /  \_______
│   /            \___
│  /                 \___
└──────────────────────────── Epoch
  warmup (1 epoch)  cosine decay
```

- **Warmup**: LR linearly increases from 0 → base_LR over 1 epoch. Prevents large gradient updates from destroying pretrained weights at the start.
- **Cosine decay**: LR smoothly decreases to near zero, allowing fine-grained convergence without oscillation.

Callbacks

Callback	Monitor	Purpose
`ModelCheckpoint`	`val_accuracy`	Save best model to disk
`EarlyStopping` (patience=7)	`val_accuracy`	Stop if no improvement

Callback	Monitor	Purpose
`ReduceLROnPlateau` (patience=4)	`val_loss`	Emergency LR reduction
`CSVLogger`	—	Log all metrics per epoch

---

8. Grad-CAM Explainability

File: `utils/gradcam.py`

Gradient-weighted Class Activation Mapping (Grad-CAM) produces a heatmap showing **which spatial regions of the MRI the model focused on** to make its prediction.

How It Works

1. Build a sub-model that outputs both the last conv layer's feature maps AND the final predictions
2. Record gradients of the predicted class score w.r.t. the conv feature maps using `tf.GradientTape`
3. Global-average-pool the gradients → one importance weight per feature map channel
4. Weighted sum of feature maps → raw heatmap
5. Apply ReLU (keep only positive contributions), normalize to [0,1]
6. Resize heatmap to input image size, colorize with TURBO colormap
7. Blend over original image using `cv2.addWeighted`

Auto-Detection of Last Conv Layer

```
def get_last_conv_layer_name(model):
    # First pass: top-level layers
    for layer in reversed(model.layers):
        if isinstance(layer, (Conv2D, DepthwiseConv2D, ...)):
            return layer.name
    # Second pass: nested sub-models (EfficientNetB4 is wrapped)
    for layer in reversed(model.layers):
        if hasattr(layer, 'layers'):
            for sub_layer in reversed(layer.layers):
                ...
```

This handles EfficientNet's nested structure — the backbone is a sub-model, so its conv layers are not at the top level. No hardcoded layer names needed.

Why TURBO instead of JET?

`cv2.COLORMAP_TURBO` is perceptually more uniform than the classic `JET` colormap. JET has a misleading bright-green region in the middle that makes low-activation areas appear visually prominent. TURBO provides a clearer low→high gradient.

---

9. Flask Web Application

File: `app.py`

Routes

Route	Method	Purpose
`/`	GET, POST	Landing page with mode selection
`/predict`	GET, POST	Upload form + prediction result

Prediction Flow

```
User uploads MRI image
        │
        ▼
load_and_preprocess_image()
  - PIL.Image.open() → convert("RGB")
  - resize to (380, 380)
  - /255 → float32 [0,1]
  - apply_clahe()
  - zscore_normalize()
  - np.expand_dims → (1, 380, 380, 3)
        │
        ▼
make_gradcam_heatmap()
  - single forward+backward pass
  - returns heatmap + predicted_idx + pred_probs
        │
        ▼
overlay_heatmap()
  - TURBO colorize
  - cv2.addWeighted blend (alpha=0.45)
        │
        ▼
render result.html
  - label + confidence
  - per-class probability breakdown
  - original image + Grad-CAM overlay (base64 encoded)
```

Model Loading

- Prefers `models/best_model.keras` (new Keras v3 format)
- Falls back to `models/tumor_classifier.h5` (legacy format)
- Loads `models/label_map.json` for index → class name mapping
- Gracefully handles missing model with a 503 response

Binary vs Multi-Class Mode

The app supports two display modes selectable on the landing page: - **Multi-class**: shows all 4 class probabilities - **Binary**: collapses to “Tumor” / “No Tumor” for simpler interpretation

---

10. Evaluation & Metrics

File: `utils/evaluation.py`

Outputs Generated

File	Description
`reports/class_distribution.png`	Bar chart of images per class per split
`reports/val_confusion_matrix.png`	Raw counts + row-normalized heatmap
`reports/test_confusion_matrix.png`	Same for test set
`reports/val_roc_curves.png`	Per-class one-vs-rest ROC curves with AUC
`reports/test_roc_curves.png`	Same for test set
`reports/val_classification_report.txt`	Precision/Recall/F1 per class
`reports/test_classification_report.txt`	Same for test set
`reports/training_history.png`	Accuracy + loss curves across all 5 phases
`logs/training_log.csv`	Per-epoch metrics for all phases

Why Macro AUC?

Accuracy alone is misleading when classes are imbalanced. Macro AUC averages the one-vs-rest AUC across all 4 classes, giving equal weight to each class regardless of its size. This is the primary performance metric for this project.

Test Set Rule

> **Evaluate on the test set only once, after all training and hyperparameter decisions are finalized.**

The test set is the true held-out estimate of real-world performance. Running it multiple times and adjusting the model based on test results is data leakage.

---

11. Bugs Fixed & Changes Made

Bug 1 — `InvalidArgumentError: not enough values to unpack (expected 3, got 1)` in `apply_clahe`

Location: `utils/augmentation.py:45` | `notebooks/train_v2.ipynb` Cell 6

Root Cause: Some MRI images in the dataset are saved as single-channel (grayscale) PNG files. When loaded by TensorFlow without `color_mode='rgb'`, they arrive as `(H, W, 1)` or `(H, W)` tensors. The original `apply_clahe` called `cv2.split()` directly — on a 2D array `cv2.split` unpacks rows rather than channels, returning hundreds of values instead of 3.

```
# ORIGINAL (broken):
r, g, b = cv2.split(img_uint8)   # ValueError on grayscale
```

Fix 1 — Source fix in `make_raw_dataset` (`train_v2.ipynb` Cell 3):

```
# FIXED: force RGB at load time
tf.keras.utils.image_dataset_from_directory(
    ...,
    color_mode='rgb',    # ← added this
)
```

Fix 2 — Defensive guard in `apply_clahe` (`utils/augmentation.py`):

```
# FIXED: handle any channel count before splitting
if img_uint8.ndim == 2:
    img_uint8 = np.stack([img_uint8] * 3, axis=-1)
elif img_uint8.ndim == 3 and img_uint8.shape[2] == 1:
    img_uint8 = np.concatenate([img_uint8] * 3, axis=-1)
elif img_uint8.ndim == 3 and img_uint8.shape[2] != 3:
    img_uint8 = np.stack([img_uint8[:, :, 0]] * 3, axis=-1)

# Now safe to split
channels = cv2.split(img_uint8)
enhanced_channels = [clahe.apply(ch) for ch in channels]
```

Fix 3 — Defensive guard in `mri_augment` (`utils/augmentation.py`):

```
# FIXED: convert at the top of the pipeline before CLAHE is called
if image_np.ndim == 2:
    image_np = np.stack([image_np] * 3, axis=-1)
elif image_np.shape[-1] == 1:
    image_np = np.concatenate([image_np] * 3, axis=-1)
```

Why three fixes instead of one? - Fix 1 prevents the problem in the training pipeline (most common path) - Fix 2 protects `apply_clahe` when called from Flask inference (which uses `PIL.Image.convert("RGB")` and is already safe, but this makes the function contract unambiguous) - Fix 3 protects the entire `mri_augment` function against any future caller that passes a grayscale array

---

Why the Original Model Had ~30% Accuracy

The original `train.ipynb` applied `Rescaling(1/255)` **both** in the `.map()` preprocessing **and** inside the model definition. This divided pixel values by 255 twice, giving the backbone near-zero inputs (`≈0.0–0.004`). EfficientNet's pretrained features are calibrated for inputs in the ImageNet range — completely zeroing them makes all 430 ImageNet feature detectors useless, leaving the model to train from scratch on a tiny dataset.

`train_v2.ipynb` fixes this by: - Doing all preprocessing in the data pipeline (`mri_augment` → CLAHE + Z-score) - Having **no** `Rescaling` layer inside the model - Receiving float32 values in `[0, 1]` with proper MRI-specific normalization

---

12. How to Run

Prerequisites

```
Python 3.10+
pip install -r requirements.txt
```

Key dependencies: `tensorflow`, `flask`, `opencv-python`, `pillow`, `scikit-learn`, `matplotlib`, `seaborn`, `numpy`

Step 1 — Train the Model

Open `notebooks/train_v2.ipynb` and run all cells top-to-bottom.

Expected outputs after training: - `models/best_model.keras` - `models/label_map.json` - `models/model_info.json` - `reports/` — confusion matrices, ROC curves, training history

Step 2 — Run the Web App

```
python app.py
```

Navigate to `http://localhost:5000`, upload an MRI scan, and click Predict.

Data Structure Required

```
data/
├── train/
│   ├── glioma/
│   ├── meningioma/
│   ├── no_tumor/
│   └── pituitary/
├── val/
│   └── (same 4 subdirectories)
└── test/
    └── (same 4 subdirectories)
```